One Name for Schema, Hierarchy and Access Control on a Replicated Log

Oscar Martinez
MOG Systems, Inc.

Abstract

This paper presents a state primitive in which schema validation, hierarchical routing, and access control are relations over a single key: the five-token subject `tenant.<tenant>.dag.<family>.<id>`. The family is a total function of the record's schema identifier. The stream that stores a family is bound to that family's branch of the subject tree. Each identity's grant is a list of patterns over the same tree. Evaluating an incoming write is a fixed sequence of local lookups on one string: derive the subject, match the grant, find the stream, validate the record. No step traverses a graph. The three lookups on the name take about 225 nanoseconds together on a laptop; schema validation takes the rest, about 15 microseconds for a 726-byte record with 21 fields.

This paper states where each of the structures physically lives and how each changes. It proves the bound on the check. It answers the three questions a systems reader will ask of a shared key: how deep a hierarchy the check must walk, what a structural change locks, and whether writers that race or partition converge. Each answer is a proof, or a boundary of the claim stated as a property with its consequence. It reports latency measured on one machine against the same checks made as calls to Postgres, Valkey and an HTTP schema registry. Policy written on a type name is not new: Hermes placed a type's policy at the node its name hashes to, Confluent's registry derives schema subjects from record types and refuses a wrong type at the broker, and NATS already binds streams and permissions to the same subjects. What this paper claims is the combination: a family derived from an immutable schema identifier that fixes a durable per-tenant stream, an unregistered type with no stream to land in, a fixed depth of five tokens, and the registry held as a family on the same log.

1. Introduction

A common stack makes three network calls to three systems before it accepts one write: a schema registry for the record's schema, an authorization service for the writer's grant, and a database for the stream the record belongs in. This system makes schema validation, hierarchical stream routing, access grants and Postgres row-level security relations over one five-token key, `tenant.<tenant>.dag.<family>.<id>`, so each is a lookup where the name already is: the first three in the writer's process and again at the broker, the row policy inside Postgres as it reads a row. Because the checks on a write are lookups on that one string, they run in the writer's process with no network call: the three lookups on the name take 225 ns (72 ns to derive the family and the subject, 66 ns to match the grant trie, 87 ns to find the stream), local JSON Schema validation of a 726-byte record adds 15.2 µs, and eighteen workers on one 18-core machine evaluate 558,484 writes per second over a mixed workload of twenty actions.

Figure 1 follows one write from its writer to the read model.

Conventional distributed systems treat schema validation, hierarchical routing, and access control as separate infrastructure tiers, requiring multiple network hops to verify a single state transition. This paper introduces a unified state primitive that collapses the schema registry, the subject tree, and identity grants under a single canonical key. Under this invariant, validating data structure, resolving namespace hierarchy, and enforcing permissions converge into a single, bounded local operation.

The shared key is the five-token subject name. The schema registry, the stream topology, the grants, and the read-model row policies are each indexed by that dotted name or one of its tokens, and each is evaluated where the name already is: the registry and the grants in the writer and the broker, the row policy inside Postgres as it reads the row. They form one operation because each is a local lookup on that name, and none of the lookups requires a network call to another service. Section 2.6 states what is unified and what remains physically separate.

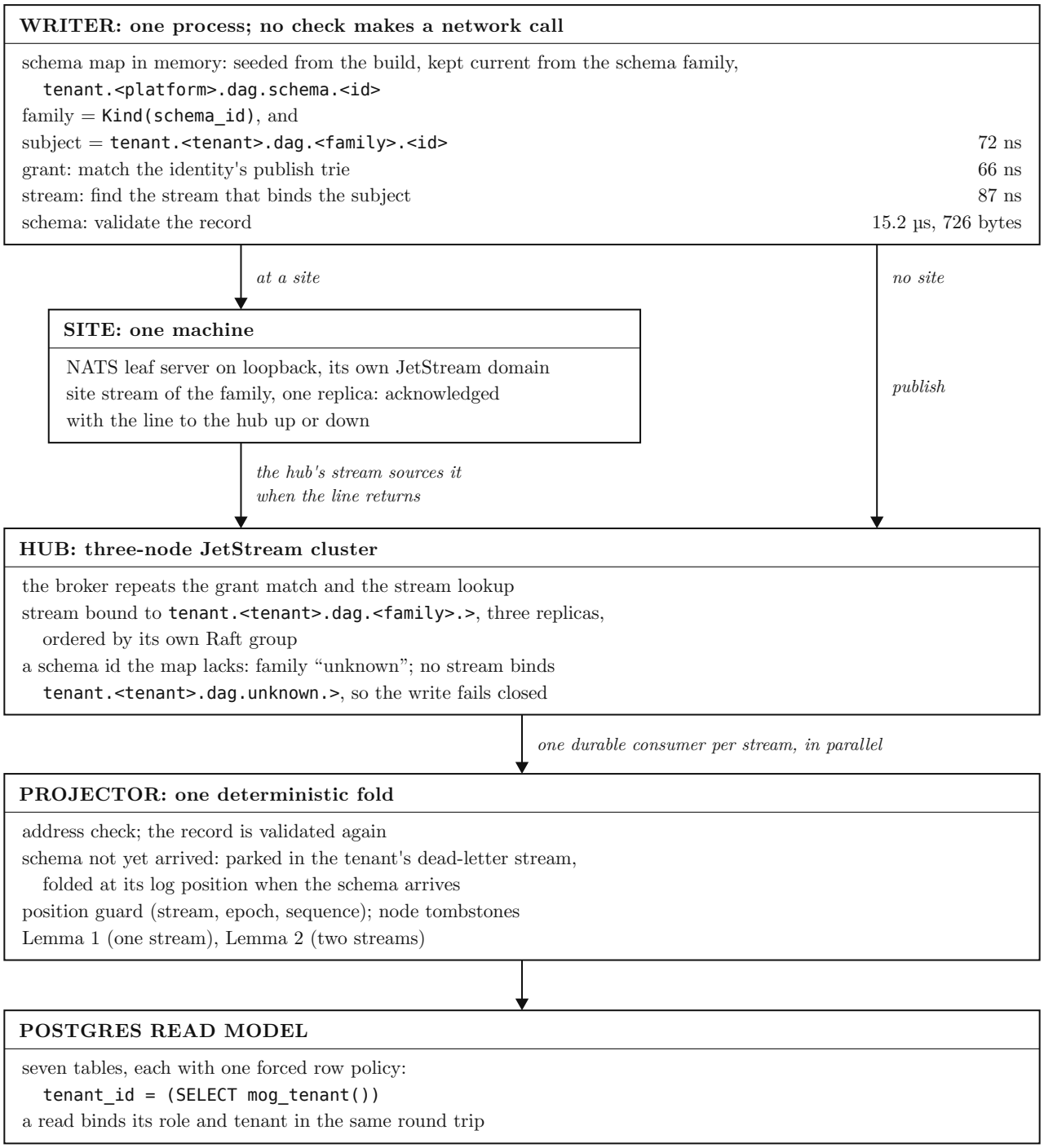


Figure 1. The path of one write. The writer makes every check before it publishes (section 3.1). At a site, the writer publishes to a NATS leaf server on its own machine, which acknowledges the record on the site's own stream, and the hub's stream of the family sources it when the line returns (section 10). At the hub the broker repeats the grant and stream checks. A schema identifier the map lacks yields the family `unknown`, which no stream binds, so the record has nowhere to land (section 2.2). The projector folds each stream in its order into Postgres. It parks a record whose schema has not arrived and folds it at its log position when the schema does, and the position guard and the tombstones keep a late fold from overwriting a newer row (sections 5, 6.4 and 6.5). A read binds the reader role and the tenant in the same round trip as the query (section 8).

The system is the ledger under the one described in the companion paper [1]. I built it and I run it. Records are JSON documents. They are stored in NATS JetStream [2], a message broker that keeps every message in ordered, replicated streams. A program called the projector folds the streams into a Postgres read model. Every statement below was checked against the implementation.

None of the parts is new. Subject tries, schema registries and access rules on topic names are all old, and writing access rules on the type name itself is older still: Hermes placed a type's access policy at the node its type name hashes to, and EventStoreDB matches stream policies on names that carry the type (section 7). What is new is not type-addressed policy on its own but four constraints held as one invariant. First, the writer does not choose the family: it is a total function of the immutable schema identifier, and a record whose subject names another family or another id is refused. Second, the broker's stream table is the set of valid types for each tenant, so a family the map does not know, or one the tenant's kind does not carry, is refused by the absence of a stream, with no call outside the broker. Third, the subject is fixed at five tokens, which bounds both the walk of the grant trie and the stream lookup by a small constant, whatever the depth of the graph in the data. Fourth, the schema registry is itself a family on the same replicated log, and writers and the projector can deploy in either order: a record whose schema has not arrived is parked at its log position and folded at that position when the schema arrives, and the position guard and the tombstones, both defined in section 6.4, keep a late fold from overwriting anything newer.

Where the claim ends matters as much. The three structures are one naming, not one store: changing a family, a stream or a grant takes three changes in three places by three mechanisms. The hierarchy the check reads is two levels deep, tenant and family; anything deeper lives in edge records, which the check never reads. A revoke takes effect in each layer at that layer's moment, not across all of them at once. A retracted node's id never takes an edge again, because nothing orders a message on one stream against a message on another. Replay shows the rebuild contained in the live model, by id, not that the log explains every live row. A writer granted a family is trusted within that family. Section 11 states these boundaries, the trust model they rest on, and what each one costs.

Section 10.4 sets out the uses beyond the studio that the primitive is designed for, and section 12 the measurements the claim still needs at larger scale.

1.1. Why NATS JetStream: native subject binding, leaf nodes, and skipping the schema registry

NATS already binds two of the four relations to one subject tree. A JetStream stream is declared with the subjects it keeps, and no two streams may keep the same subject. A permission is an allow list of subject patterns per identity, matched by the same subject trie the server routes with [4], [26]. So the stream topology and the grants are stock NATS, written on the same names. That half of the claim is not new, and it is why I chose this broker.

What I add brings the schema into the same tree. The fourth token of every subject is `Kind(schema_id)`, a total function of the record's immutable schema identifier, so the identifier, not the writer, decides which stream a record goes to (section 2.2). The schema registry is itself a family on the log, under `tenant.<platform>.dag.schema.<id>`, and every writer and the projector hold it as a map in memory fed by a subscription to that stream (section 2.3). The log therefore carries the definitions of its own types, and a record's type decides where it is stored. An identifier the map does not know yields the family `unknown`, and no stream binds `tenant.<tenant>.dag.unknown.>`. The broker itself does not read schemas: the writer validates before it publishes, and the projector validates again before it writes a row (section 3.4).

The usual alternative is Kafka with an external schema registry and ACLs [14], [15], [16]. There a topic exists whether or not any schema maps to it. The registry is a separate service with its own store, reached over HTTP, and the ACLs are a third list, written on topic names rather than record types. Confluent's broker-side validation can refuse a record whose schema is not registered under the topic's subject, but the broker asks the registry to decide, and the topic, the schema and the grant are still three things kept in three places. Here they are one name. Section 7 sets the two side by side with the other systems, and section 8 measures the local checks against the same checks made as three calls.

The last reason is the edge. A NATS server can run as a leaf node of another cluster, with JetStream in a domain of its own [35], [36], and a stream on one server can source the messages of a stream on another, from where it left off [37]. A site in this system is one such server on the site's own machine. It listens on loopback only and holds a one-replica stream for each family the site writes. A writer at the site gets its acknowledgement from that stream whether or not the line to the hub is up, and the hub's stream of the family sources the site's stream when the line returns, in the site's order. A site needs one machine, where a cluster of its own would need three, and the checks it runs are the same local checks (section 3.5). Section 10 specifies the site and measures its disconnection and reconnection.

2. The record layout

2.1. The record

Every record is one envelope. A record is a node or an edge. Its schema identifier decides which, through the family (section 2.2).

Column	Type	Node	Edge	Meaning
`node_id`	`nd_[A-Za-z0-9_-]{6,}`	required	absent	The node's identity
`edge_id`	`ed_[A-Za-z0-9_-]{6,}`	absent	required	The edge's identity
`schema_id`	`sch_<Name>_v<N>`	required	required	The registry key of the record's schema
`tenant_id`	slug	optional	optional	The writer's claim of tenant; the subject decides (section 2.5)
`created_at`	RFC 3339	optional	optional	When the writer made it
`content_hash`	`sha256:<64 hex>`	optional	no	Idempotency key over the underlying bytes
`source_id`, `target_id`	node ids	no	source required	The edge's endpoints
`payload`	object	required	optional	The body its schema governs

The envelope checks `tenant_id` as a slug pattern and lists no tenants. Which tenants exist is decided where a record lands: a record published under a root with no stream has nowhere to land, and the projector takes the tenant from the subject (section 2.5). The tenant set is the platform root's tenant records. The projector, the replay tool and the bootstrap read it from that stream at boot and follow it by subscription, with the build's list as the seed only. A tenant record appended at run time has its streams bound by the projector at once on the record's arrival, and at the latest by its next rescan, 30 seconds after the last, with no restart.

An id is either content-addressed or derived from the fields that define the record, such as the tenant's id seed, a type, a subject and a key. A tenant's id seed is its slug unless its tenant record names another; a tenant that takes over another tenant's records keeps that tenant's seed, so no derived id moves. The derivation is SHA-256 over the fields joined by a zero byte, truncated to 15 bytes and base64url-encoded: 120 bits in 20 characters, always one subject token. Within one tenant and family, an even chance of two ids colliding takes a little over 2^{60} records, more than a quintillion, and at a billion records the chance is about 3.8 in ten billion billion, far below the chance of an undetected memory error on the host. The same record written twice has the same id. Not every id includes the tenant (section 6.4).

2.2. The subject

A record is published to exactly one subject:

```
tenant . <tenant> . dag . <family> . <id>
```

The subject has five tokens. The id alphabet contains no `.`, `*` or `>`, so an id is always one token. The family is `Kind(schema_id)`, a total function of the schema identifier resolved from the local schema registry map (seeded from a compiled table for built-in schemas), one lookup in O(1). An identifier the map lacks yields the family `unknown`. The writer does not choose a record's family. A record's subject must be its own address: exactly five tokens, `tenant` first and `dag` third, the tenant token the tenant of the stream the record was read from, `Kind(schema_id)` in the family token, the record's own id in the last, and that family the family of the stream that holds it. At the

broker the writer does choose the subject it sends, within its grant; the derivation is enforced after the log, by one function in the reference implementation that the projector's live handler, its pass over consumer gaps, the replay tool and the readdress all call. The projector refuses any other subject, terminates the record rather than parking it, and raises it as an exception, and the replay tool refuses it and counts it.

A family has at most one stream per tenant, bound to `tenant.<tenant>.dag.<family>.>`. A tenant's record names its kind, and the bootstrap identity creates one stream under the tenant's root for each family that kind carries, and no other. A family's streams follow its schema record the same way: the append of the schema, and every bootstrap after it, create the family's stream under each recorded tenant of the kinds the record names, and the projector binds it when the schema arrives. Every record stream is named for its family. A stream the bootstrap creates is capped at 8 MiB for a mock tenant, a tenant the test harness uses and no real work writes, and 256 MiB for any other; the first tenant's twenty-four streams declared in the infrastructure definitions keep the caps declared there. A record is never dropped. Every record stream, and the platform root's tenant, user and schema streams, discards new messages at its cap rather than old ones, and denies delete and purge, so a stream at its cap refuses the next write with an error at the writer and keeps everything it holds. A fill metric per record stream raises an alarm at 80 percent of the cap, so a filling stream is caught and its cap raised well before it refuses anything. A write fails on the storage a broker uses, not on what the caps reserve, so a stream with no cap of its own is bounded by its brokers' storage. Each broker's storage used, the bytes of every stream it holds a copy of, is measured against its limit, and an alarm is raised when the fullest broker reaches 70 percent or the measure goes missing, before any write fails on it. "Append-only" in this paper means exactly that: no edit, no delete, no purge. The one stream that deletes is a tenant's dead-letter stream, which is not a record stream: the projector removes a letter once it has applied it (section 5). No stream binds `…dag.unknown.>`, so a record whose identifier the map lacks has nowhere to land, and neither has a record of a family its tenant's kind does not carry. Work queues are not record streams. Task messages ride their own subjects, `…dag.task.<queue>.<arm>`, and no record stream binds them. The reference implementation has 57 schemas across 28 families.

The stream table also holds the streams for worker facts and presence, under `…dag.ingress.>`. Their subjects do not overlap the record branches.

2.3. The schema registry

The registry is a map from schema identifier to a compiled JSON Schema (draft 2020-12 [3]), held in every process that validates. The lookup is a map read, never a call. A schema is immutable per identifier: the identifier carries its version, and a change is a new identifier.

The map is kept current from the schema stream. A schema is a record of its own family, on the platform root rather than a tenant's: `tenant.<platform>.dag.schema.<id>`, with the id derived from the schema identifier so that one identifier has one subject. The record carries the identifier, the JSON Schema document, the family its records go under and the tenant kinds that carry the family. The grants name two writers for the stream, the bootstrap identity and the emergency identity of section 2.4, and the bootstrap appends an identifier once. The same definition again changes nothing. Another document, family or kinds under the identifier is refused as a conflict. Of two writers racing to the first append, the broker refuses the second, because the publish expects an empty subject. The append returns only after the broker's acknowledgement and a read-back, and the identifier is usable from that moment. A build that edits a schema in place fails its bootstrap step by the identifier's name.

Every binary requires only two schemas at build time (the envelope and the schema record's own), though the reference build also embeds the baseline snapshot of 57 schemas as a cold-start floor, so the process can boot and validate even if the schema stream is unreachable. Because every schema identifier is immutable (`sch_<Name>_v<N>`), an embedded schema and its stream record are identical by construction, and pre-seeding known immutable schemas in the binary is just a cold-start cache, not a frozen registry, because new schemas and families still enter the map dynamically from the stream at runtime. At boot, each writer and the projector seed their in-memory map from this embedded snapshot, read the schema stream to load any schemas added since the build, and hold an ordered consumer on the stream for new arrivals.

`Kind` reads a family map seeded the same way. The snapshot carries the family of each embedded schema, keyed by its identifier, and the stream adds the family each schema record names. A lookup takes no lock; a registration copies the map and swaps it in whole. A schema appended at run time therefore validates records, and resolves its family, in a running process with no restart. The bootstrap appends every embedded schema to the stream and refuses a document that differs from the one the stream holds, so a build that edits a schema in place fails that step; that is what makes the two identical. A reader can meet a differing stream document only when a build skipped its bootstrap, and it then keeps the embedded one and logs the difference once. For an identifier from the stream, the first definition is the definition. A process that meets an identifier its map lacks re-reads the stream once, bounded to five seconds, before it refuses the record. The re-read closes the window between an append and a reader's subscription, and binds a stream that was absent at boot. A test fails the build when an embedded schema has no family in the snapshot, or the snapshot names a family for a schema the binary does not embed.

Why the snapshot is not the registry. A registry that is only the binary's snapshot fails on three counts. On evolution, a schema change becomes a build and a deploy of every writer and the projector, in an order nothing enforces. On tenancy, a family no binary embeds cannot reach a tenant without a deploy. On vocabulary, "registry" means versioned lookup at run time, and a frozen map invites the reply that the invariant of section 2.6 holds only because nothing can change. The primitive needs two things of a schema: that it resolve locally, in the process that evaluates the write, and that it be immutable per identifier. Neither needs a build. With the stream, a family no binary embeds arrives by one schema record naming its kinds, and the bootstrap creates its streams under every tenant of those kinds (section 2.2).

The writer validates before it publishes. The projector validates again before it writes a row (section 3.4).

2.4. The grant tuple

A grant is a set of tuples:

```
(identity, verb, pattern)        verb in {publish, subscribe}
```

A pattern is a subject in which a token may be `*`, which matches exactly one token, or which may end in `>`, which matches one or more tokens. The tuples are written per identity in the infrastructure definitions and rendered into each server's configuration as `permissions: { publish: { allow: [...] }, subscribe: { allow: [...] } }`. Every identity receives its replies on its own reply prefix, which the rendering appends to its subscribe list, and no other identity can read them. The cluster's grants have no deny lists; a site's leaf has deny lists of its own (section 10.2).

The cluster has 23 identities. These are the roles that reach a tenant's tree:

Role	Publish patterns	What the patterns cover
The authority and the projector	923	`tenant.<tenant>.dag.>`, the dead-letter subjects of the other roots, one task subject, and 918 JetStream API subjects naming its streams and consumers
The on-premises GPU worker	45	two record families, its own presence and fact subjects, and the JetStream API subjects of its work queues and of its site's leaf
The cloud GPU worker	15	the same, for the failover arm, with no leaf
Two browser workers	13	its fact subjects and named durable consumers only; no record family
A third browser worker	19	the same, plus two more work kinds
Three editing workstations	46 to 49	the same two record families and the validation family for QC findings, one named durable per work kind
The bootstrap identity	many	the platform root's schema, tenant and user subjects, and the JetStream stream administration subjects
The emergency identity	1	`>`

Two identities stand outside the per-family grants. One, the bootstrap identity, administers streams and writes the platform root; it is how a family, a tenant or a schema is added, and it holds the stream administration

subjects no service identity has. The other may publish and subscribe to everything, and exists for emergencies. The counts are of the patterns as written on one tenant's root.

2.5. The row-policy tuple

The read model holds seven tables: the node table, the edge table, the tombstone table and four derived tables. Each row carries a tenant:

```
(row, tenant_id)      tenant_id NOT NULL
```

Each table has one forced row-security policy:

```
USING (tenant_id = (SELECT mog_tenant()))  WITH CHECK (tenant_id = (SELECT mog_tenant()))
```

`mog_tenant()` is a function that reads the transaction's tenant setting, `mog.tenant`, and raises an error when it is missing or empty. So the policy protects against one thing precisely: a read that forgets to bind a tenant fails rather than returning every row. It does not protect against a read that binds the wrong tenant. `mog.tenant` is a session setting that any holder of the reader credential can set to any value, so the boundary that a reader sees only its own tenant is enforced by the service that binds the tenant from the caller's credential (section 3.4), not by the policy; the policy is the backstop that makes a missing binding fail closed. Reads run as a reader role with `NOBYPASSRLS` that has `SELECT` on the read-model tables and nothing else. The role is a login role, since the read-model services connect as it directly; it is given no other privilege. A read transaction sets both the role and the tenant with `set_config(…, true)`, and both revert at commit. The projector is the only writer. It writes the node, edge and tombstone tables as a role that bypasses the policies, and it writes the four derived tables one tenant at a time as a writer role with no bypass, so the same policy binds those writes.

The row's tenant is the tenant of the subject the record arrived on, not the tenant its envelope claims. On any other tenant's subject, an envelope that names a different tenant is refused and not projected. On the root tenant's subject the disagreement is logged and the row takes the subject's tenant. The grant decides who may publish on a subject, so the subject is the stronger claim. Every table is keyed by tenant and id, so an id that two tenants write is two rows (section 6.4).

2.6. One naming, not one storage

Read the four structures as relations on one key, the subject:

```
Registry(schema_id -> schema)            with  family = Kind(schema_id)
Tree(family -> stream)                   a stream binds tenant.<t>.dag.<family>.>
Grant(identity, verb, pattern)          pattern is a subject pattern over the same tree
Policy(row -> tenant_id)                 tenant_id = token 2 of the subject the row came from
```

Each relation is evaluated on the subject or on one of its tokens. That shared key is the invariant. Where each relation physically lives:

Relation	Where it lives	Who reads it	How it changes
Registry and `Kind`	A map in every writer and the projector, seeded from the binary's snapshot and kept current from the schema stream	The writer, before publishing; the projector, before writing a row	An append to the schema stream by the bootstrap identity: a new identifier, never an edit
Tree	Stream configurations on the broker, one per family of the tenant's kind	The broker, routing each publish; the projector, binding	Created by the bootstrap from a tenant or schema record; twenty-four streams are also declared in the infrastructure definitions
Grant	The broker's configuration file	The broker, on every publish and subscribe	A configuration change, which each broker in turn reloads in place
Policy	Postgres catalog, one policy per table	Postgres, on every row a read touches	Reinstalled idempotently on each projector boot, under a bounded lock

So the claim holds as one naming, not one storage. Section 9 sets out what one storage takes and what it costs.

3. The evaluation function

3.1. A write

An incoming write is an identity, the tenant it writes for, and the bytes of a record.

```
evaluate(identity, tenant, raw):
  1  e        <- decode the envelope fields schema_id, node_id, edge_id from raw
  2  family   <- Kind(e.schema_id)                        one lookup in the registry map
  3  subject  <- "tenant." + tenant + ".dag." + family + "." + id
  4  if no publish pattern of identity matches subject:  refuse not_granted
  5  stream   <- the stream whose subject filter matches subject
     if none:                                            refuse no_stream
  6  if raw does not conform to registry[e.schema_id]:   refuse schema
  7  allow; the record is appended to stream
```

Steps 4 and 5 use the broker's own subject index (section 3.2). The order of 4 and 5 is the broker's: it refuses an ungranted publish at the connection, before it routes the subject to a stream. The order changes which refusal is reported, never whether one is.

The function is implemented in the benchmark harness. A test publishes each workload action as raw bytes, with no check first, to a broker configured with the same grants and streams. It confirms that the broker reaches the same verdict on steps 4 and 5 for every action. The broker refuses an ungranted publish with a permissions violation. It answers a publish to an unbound family with no acknowledgement from any stream. It stores a record that fails its schema, because the broker does not read schemas. Schema enforcement belongs to the writer, and the projector re-checks it (section 3.4).

The grants decide which writers there are (sections 2.4 and 4). Most workers publish only facts, on their tenant's ingress subjects, and the authority turns facts into records. The authority and the projector hold publish on a tenant's whole tree. The GPU workers hold publish on two families, assets and derivation edges, and the editing workstations on those two and the validation family, for their QC findings. The bootstrap identity writes the platform root's schema, tenant and user records, and the emergency identity holds everything (section 2.4). The bootstrap identity also administers streams, and it is the identity that runs a readdress (section 6.7). So the identities that can publish a record family are the authority and the projector, the two GPU workers, the editing workstations, and the two privileged identities, the bootstrap and the emergency one; no service identity beyond those holds publish on a record family. Every record stream denies delete and purge in its own configuration (section 2.2), so the log is append-only for every identity: a stream administrator can change a stream's configuration, but no identity removes a record from a record stream without first changing that configuration, which is itself a change to the infrastructure definitions.

3.2. The bound

Claim. For a record of size n, the evaluation does work bounded by a constant that depends on the grant list, the stream table and the schema, plus validation work in n. Nothing in it depends on the number of records, tenants or identities stored.

Proof, step by step.

1. *Decode.* One pass over n bytes to read three fields. The validator decodes the record again in step 6, so the record is parsed twice; the envelope decode is about 1.2 µs of the whole evaluation below, counted in neither the lookups nor the validation row.
2. *Kind.* One lookup in the registry map, a hash map keyed by the identifier: O(1). Measured alone, it takes 3.8 ns with no allocation.
3. *Subject.* Concatenation of five tokens. Linear in the id length, which the schema caps by pattern.
4. *Grant match.* An identity's patterns are held in a trie with one level per token. Each level has a map of literal tokens, one `*` child and one `>` child [4]. Matching a subject of k tokens descends the literal child and the `*` child at each level and collects any `>` child it meets. Every visited node is a prefix of some pattern. So the work is bounded by the total number of tokens across the identity's patterns, and by 2^k paths of length k. Record

subjects have $k = 5$. The largest grant, the authority's, has 923 patterns, 918 of them JetStream API subjects. A record subject's first token is `tenant`, so the match never enters the branch under any other first token. The broker also caches each connection's verdict per subject, up to 128 subjects.

5. *Stream lookup.* The same trie over the stream filters. The same bound.
6. *Validation.* The schema is compiled once. Validation walks the record against it. Patterns are Go regular expressions, which run in time linear in their input. Some schemas use `uniqueItems`, which is quadratic in the length of that one array. The broker refuses any message over 8 MB. So validation is bounded by a polynomial of degree at most two in n, with n capped. On the two records measured it runs at 21 ns per byte for 726 bytes and about 25 ns per byte for 208 bytes.

No step reads a record other than the one being evaluated. No step follows an edge. The bound holds whether the log holds a hundred records or a hundred million. ■

Measured. On an Apple M5 Max, Go 1.27.1, `go test -bench` over five runs, the median shown (the benchmark report [5], which also gives the range over a second set of five runs):

Step	Time	Allocations
`Kind` and the subject	72 ns	4
Grant match	66 ns	0
Stream lookup	87 ns	0
Schema validation, a 726-byte record with 21 fields	15.2 µs	375
The whole evaluation, the same record	17.4 µs	380
The whole evaluation, a 208-byte record with 8 fields	5.5 µs	116

The three lookups on the name cost about 225 ns together. Validation is about 87 percent of the 726-byte record's evaluation. The grant-match figure is measured on one worker identity's grant, which has nine patterns, not on the authority's 923. By the argument of step 4 the match is a hash lookup per token level and does not grow with the number of patterns, so the larger grant measures the same within noise; the point is that the figure does not depend on the grant's size, and the 923-pattern case is not separately tabulated here.

3.3. A read of the log

A subscription is a pair: an identity and a subject or filter. The broker allows it when a subscribe pattern of the identity covers it. That is step 4 with the subscribe patterns, and the same bound holds. An identity with a narrow grant can only create its own named durable consumers. The create subject carries the consumer's filter, so the grant pins the consumer to its own subjects. The shape is `$JS.API.CONSUMER.CREATE.<stream>.<consumer>.tenant.<tenant>.dag.task.<queue>.<arm>`.

3.4. A read of the read model

A read by id under the policy does one index lookup on the primary key and one comparison of the row's tenant with `mog_tenant()`. The function is written `(SELECT mog_tenant())`, so Postgres evaluates it once per query as an init plan, not once per row. A read of another tenant's row answers exactly as a read of a missing row does.

The projector re-validates every record before it writes a row. A record under an identifier the projector's map does not know, after the re-read, is parked in its tenant's dead-letter stream with its log position (section 5). A record under a known identifier that fails its schema is terminated, since an identifier is immutable and the record never will pass. So a granted writer that skips validation can put a malformed record in the log, and it stays there, because a record stream never drops a record and the broker does not read schemas. It never reaches the read model. A consumer that reads the log directly reads it as published: the schema guarantee holds in the read model, not in the log. The defense in the log is the grant table of section 3.1, which keeps every identity but those it names off the record families. The projector compares the subject's tenant with the envelope's (section 2.5), and checks the subject against the record's own address (section 2.2): exactly five tokens, `tenant` first and `dag` third, the tenant token the stream's own tenant, the family token `Kind(schema_id)`, the last token the record's id, and the family the family of the stream the message was read from. In the reference implementation one function makes that check, and the live handler, the projector's pass over gaps in its consumers, the replay tool and the

readdress all call it, so no path folds a record the live path would refuse. A granted writer that publishes a record under another family, under another record's id, or at any other depth, puts it in the log, and the projector refuses it there, so it never reaches the read model.

3.5. What the primitive gives

Three properties follow from the shared key. Each holds in the scope stated here and no wider.

- **Zero-hop verification.** Every check in the evaluation runs in the process that holds the record, with no call to another service. The write itself is still one hop, to the broker, and the broker repeats the grant and stream checks at no extra hop. Section 8 measures the difference between this and three calls.
- **Disconnected site execution.** The evaluator and a broker with every stream and grant run in one process. The benchmark does exactly that. A site therefore checks and accepts writes with no network. It writes to streams of its own, each totally ordered at the site, and the hub sources them when the line returns, so the argument of section 6 covers them. Section 10 specifies the site.
- **Unified auditability, of who could write.** The grants are files under version control, written from the bootstrap's output; the schemas and the records are in the log. So "who could write what, and when" is answered by the grant file's history beside the log: two sources, not one. This answers who was permitted to write, not who did: the envelope carries no authenticated publisher, and the broker does not stamp one, so the log says a record of a family exists, not which granted identity sent it.

4. Depth: the check never traverses

When hierarchy and grants share a store, a check on an object twelve levels deep seems to need twelve lookups, or a scan, to find an ancestor that grants access. This section shows that the check here never traverses, and states the limit that buys that.

The hierarchy the name carries has a fixed depth of two: tenant, then family. Grants are evaluated on the name alone. Hierarchy deeper than that is held in edge records, and edges take no part in the check.

Proof. A record subject has exactly five tokens, and the tenant and the family are tokens two and four. A grant pattern is matched token by token against those five tokens (section 3.2, step 4). Nothing in the match reads another record. So the check costs the same for a record with no lineage and for one twelve derivations from its source. The bound is the grant trie's, independent of the depth of any graph in the data. ■

What makes this possible. The system never grants access to one record by way of another. A worker may write two or three families of its tenant, or none. Fine-grained decisions are made by the authority before it writes. Most workers write no records: they publish facts on their tenant's ingress subjects, and the authority turns facts into records (section 3.1). The policy surface is coarse on purpose: tenant, family and verb.

The limit. A rule such as "a reviewer of a record may read the records it was derived from" cannot be written as a grant. It needs the derivation edges, which form a graph of any depth. Zanzibar meets this with a separate index that stores the flattened group-to-group closure, so a nested membership check becomes a set intersection [6]. This system has no such index. A rule over edges needs one of three things. Edges can be materialised into the subject, which fixes the depth. The closure can be maintained beside the log. Or the decision stays with the authority at write time, which is where this system keeps it.

Reads that walk edges are bounded by a depth cap, not by the absence of cycles. The implementation's one recursive walk over derivation edges is capped at 50 levels.

5. Change: three mechanisms, no shared lock

Data writes are continuous, identity changes are frequent and schema changes are rare. In one shared table, a migration or a reorganisation locks that table against low-latency writes. Here the three structures change by three mechanisms in three places, so they never share a lock. The cost is that a change across them is not one atomic transaction. A revoke reaches the broker and the read model at different moments. Each layer closes its own window.

Change	Mechanism	What it blocks
A record is written	An append to one stream. The stream's leader orders it through the stream's own Raft group	Nothing outside that stream
A family is added	A schema record appended to the schema stream, naming the family and the tenant kinds that carry it. The append and every later bootstrap create the family's stream under each recorded tenant of those kinds, and the projector binds it when the schema arrives. No build, no deploy, no restart	Nothing. An append to one stream and the creation of another touch no other stream
A tenant is added	A tenant record appended to the platform root by the bootstrap identity, naming the tenant's kind; the bootstrap creates one stream under the tenant's root for each family of that kind, and the projector binds them when the record arrives. No build, no deploy. The tenant's machine identities are a grant change	No existing stream. It does consume cluster capacity: one three-replica stream and one consumer per family, with storage reserved (section 11)
A grant changes	The broker's configuration. The change reaches the three servers one at a time, and each reloads `authorization` in place [7]. A change the server cannot reload, such as a listener or the JetStream domain, restarts it instead, in about two and a half minutes, most of it the drain of its connections. No configuration change replaces an instance	A reload blocks nothing; a restart moves that server's connections to the other two
A membership changes	A user record on the platform root, written by the bootstrap identity, carries the member's role in each tenant. From it, each layer takes its own form. The broker: the bootstrap emits one grant per machine identity in the broker's configuration shape, and a configuration change carries them to the servers, each of which reloads it in place. The credential: a key record carries its tenant and memberships, and a person's token carries a tenants claim that the edge forwards; every service binds the transaction's tenant from that and compares	Nothing in the read model. The row policy compares a column with a session setting, so membership never touches the tables
The row policy itself	`create policy`, which takes an exclusive lock on its table	Reads and writes of that table, for the moment of the install. The projector reinstalls the policies idempotently on each boot, taking the lock under a `lock_timeout` with retry so a deploy queues reads briefly rather than stalling on a long lock

The revoke window, layer by layer.

1. *At the broker.* On a configuration reload, the server re-authorises every live connection. It disconnects any whose user was removed, and it removes every subscription the new grant no longer allows [4]. From that moment the identity can neither publish nor receive outside its new grant. The servers take the change one at a time, so the window is the time from the edit to the last server's reload. The paper does not quantify that window beyond the restart case: a change the server reloads in place is bounded by the roll across the three servers, and a change that forces a restart takes about two and a half minutes per server.
2. *In the read model.* The tenant is bound from the credential when each read transaction begins, so an agent key, which is looked up per request, is refused at its next transaction. A person's memberships travel in a signed token, so a membership removed from the person's record takes effect only when the token is reissued, which is a longer window set by the token's lifetime, not by the next transaction.
3. *For in-flight writes.* A publish the broker accepted before the reload stays in the log. Nothing orders a revoke against a write the way Zanzibar's zookies order an ACL change against a content change, and Zanzibar calls that ordering the defence against the "new enemy" problem, applying an old access rule to content created after the rule changed [6]. The matching failures here are a write that a not-yet-reloaded server accepts after the edit, and records delivered to a subscriber during the roll before its server drops the subscription. No message carries the grant version it passed under or the identity that published it, so these writes cannot be told from the rest afterward. This system accepts that window.

Schema evolution needs no deploy order. A writer and the projector can run different builds. A change is a new identifier (section 2.3), so records validated under `v1` still validate on replay. Immutability per identifier is all the system enforces on evolution: it does not check that `v2` is readable where `v1` was, as a compatibility mode in a registry such as Confluent's would. The fold takes each record whole whatever its version, so a reader must handle every version of an identifier it meets, and a `v2` that a reader cannot read is a reader-side fault, not one the registry prevents. The discipline that keeps versions compatible is held in how new identifiers are written, not a rule the broker or the registry enforces. A record whose identifier the projector's map lacks, after the re-read of the schema stream, is parked whole in its tenant's dead-letter stream. The headers carry the original subject, stream, stream epoch and sequence, and the position is the message id, so a redelivery parks nothing new. The original is acknowledged only after the dead-letter stream acknowledges, so the record stream keeps moving. The projector drains the dead letters at its start, before it consumes records, and again each time a schema new to its map arrives on the schema stream, by the subscription or by a re-read. A pass reads every dead-letter stream, folds each dead letter whose identifier the map now holds at the letter's original position, and deletes it. Passes run one at a time, so no two overlap. A record parked because its schema arrived after the re-read therefore lands when the schema arrives, with no restart. The position guard of section 6.4 orders it against every later write of the same id, and the position in the headers is checked against the log before it is used, so a header cannot make a record newer than it was. What remains in a dead-letter stream is the records the read model lacks for want of their schema, with one exception: when the dead-letter publish itself fails, the record is terminated rather than parked, so the read model lacks it and the dead-letter stream does not hold it. The record is still in its record stream, which never drops it, so a full replay projects it; the termination is logged. The dead-letter stream is the one stream that deletes, because the projector removes each letter it has applied; it is not a record stream, and it holds only copies whose originals stay in their record streams.

Parking belongs to the projector. The writer cannot know which registry the running projector holds. A negative acknowledgement stalls the stream behind one message, and termination loses the row until a full replay. Only a record under an unknown identifier is parked. A record under a known identifier that fails its schema never will pass, and is terminated.

6. Convergence under concurrent and partitioned writers

Suppose two agents produce conflicting schema variants, or a cycle in the subject tree, while partitioned from each other. Do two nodes that process the same writes in different orders reach the same state?

The log does not diverge, because each stream is totally ordered by consensus and a partition's minority cannot write. Schemas cannot conflict at run time, and the subject tree cannot form a cycle. The read model is a deterministic fold of each stream in its order. Across streams it converges for every record that does not delete rows owned by another stream. One effect does, a retraction that deletes edges on the edge stream, and one redelivery case breaks the order inside a stream; section 6.5 shows how the fold removes both. Convergence rests on total order plus a deterministic fold, not on commutative merges.

6.1. The log

Each stream at the hub has three replicas and is ordered by a Raft group of its own. A publish is acknowledged only after a majority of the replicas hold it. In a partition, the side without a majority cannot elect a leader, so it accepts no writes. The publisher gets no acknowledgement and must retry. So under the consensus protocol no two replicas agree to different messages at one sequence number, and disagreement can arise only in what is computed from the log. The price of this choice is availability: during a partition, the minority side cannot write at all.

The crash model is worth stating, because an acknowledgement is a consensus fact, not a durability fact. A majority holding a message means a majority has it in memory and in its write-ahead path, and each server flushes to disk on its own schedule. This deployment leaves the flush interval at the broker's default rather than flushing on every write. The boundary is plain: acknowledged writes can be lost if every replica of a stream loses power within the flush window. A site stream (section 10) has a single replica with the same lazy flush, so a site that loses power can lose writes it acknowledged. The convergence results here assume the consensus

crash model, a minority of servers lost at a time with their disks intact; a tenant that needs durability against simultaneous power loss sets the flush to every write and pays the latency. The latency figures in section 8 are with the default flush.

The broker also de-duplicates by a message id the writer sets, not by the record's bytes, within a two-minute window per stream. The writers set that id to the record's own id, the asset's content hash or the edge id. So two publishes of the same id inside the window are collapsed to the first, even if the second carries a newer version: inside the window the first write wins, not the last. This matters only for a writer that republishes a changed version of one id within two minutes, and it is why the position guard below, not the broker's de-duplication, is what settles versions.

A site that writes while cut off from the hub writes to streams of its own, each ordered by the site's server, and the hub sources them when the line returns (section 10). No stream is ever ordered by two servers.

6.2. Schemas

The schema stream's writer is the bootstrap identity, and an identifier is immutable (section 2.3). A second append under an identifier with another document, family or tenant kinds is refused as a conflict, and of two writers racing to the first append the broker refuses the second. For an identifier the binary does not embed, each reader takes the first definition from the stream; for an embedded one, the two are the same (section 2.3). So two agents cannot register conflicting variants at run time: no identity but the bootstrap and the emergency one holds a grant to append, and the first definition of an identifier is the definition. The conflict is settled before either variant can reach a record stream. This rests on the first definition of an identifier staying on the schema stream, since the expect-empty-subject check is what refuses a second definition, and it does: the schema stream discards new at its cap and denies delete and purge (section 2.2), so a first definition is never removed, and a full schema stream refuses the next append rather than evicting an old one.

6.3. The subject tree

A cycle needs a node that is its own ancestor. The subject tree has no parent pointers. Its depth is fixed at five tokens, and a subject is a string, not a path through records. So a cycle in the tree is impossible by construction. Cycles among edges are possible: nothing refuses a `derived_from` edge that closes a loop. Reads that walk edges stop at a depth cap (section 4).

6.4. The fold

The projector holds one durable consumer per stream and applies each message to Postgres through one fold, which the replay tool runs as well, so the two cannot drift:

- Every node and edge row carries the position of the message that wrote it: the stream, the stream's epoch (its creation time, so a stream deleted and made again orders after the old one) and the sequence. A write applies only when its position is newer than the row's. Positions on two streams are ordered by the immutable time the broker stored each message, then by stream name. Two clocks enter this order: the broker's store time, which is fixed in each retained message and so reads the same on every projector, and the stream epoch, which is a creation time, so a stream recreated on a server whose clock is behind could order before the old one. The epoch matters only when a stream is deleted and remade, which the record streams are not in normal operation. Where a position component is missing, a row written with no stored position or a dead letter whose header time does not parse, the newer test is permissive and the last delivery applies; those rows do not carry the log-order guarantee.
- A node is upserted by its tenant and `node_id`. The row takes the incoming record whole, with two exceptions: its creation time is the earliest that any record of the id carries, and two fields of one node kind survive a later record that omits them.
- An edge is upserted by its tenant and `edge_id`, and its creation time is likewise the earliest any record of the id carries. An edge whose relation is one of the two retracting relations deletes the row with that id. A deletion writes a tombstone row at its position.

- A node marked `retracted`, with one kind excepted, deletes itself, every edge that touches it, and its derived rows, and writes a tombstone for the node. No edge write passes while either endpoint has a tombstone. The retraction and every edge write take an advisory lock on the tenant and node ids they touch, and every delete and tombstone names the tenant.
- The derived rows for a node of the first kind are recomputed from the current tables whenever that node, or a node or edge that feeds its derived rows, arrives, or an edge that fed them is removed.
- The creation time is the minimum over the id's records, applied on every write. When the position guard skips an older record, a second statement in the same batch still lowers the creation time to that record's, at no extra round trip. A record at or below the id's tombstone belongs to an incarnation before the deletion and is left out, so a node written again after a retraction keeps the time it was written again.

Write S for the read model and *apply(S, m)* for one message's effect.

Lemma 1 (one stream). Every effect is keyed by an id carried in the message. Within a tenant, every version of an id maps to one family, so to one subject branch and one stream, and the projector refuses a record published off that address (section 2.2). Where a legacy id's versions were written under two families, the position guard orders them by the immutable time the broker stored each message, then by stream name (the first rule above). That store time, fixed in each retained message, is the main place a clock enters the fold (the stream epoch is the other, and matters only for a recreated stream), and it reaches only ids written under two families: every other id has one family and one stream, ordered by that stream's own positions, and since each store time is fixed in its retained message, every projector and every replay reads the same times and folds the same row, however far apart the brokers' clocks were when the messages were stored. Ids are not all tenant-scoped: the content-addressed asset id and the annotation ids hash no tenant, so the same file under two tenants has one id on two tenants' streams. Every read-model table is keyed by tenant and id, and every statement of the fold names the tenant, so those are two rows, each its tenant's, and the lemma holds across tenants. A write applies only at a position newer than the row's, and a deletion leaves a tombstone at its position, so a delivery older than the row or the tombstone changes nothing. The row's main payload is therefore a function of that id's messages in log order, whatever order they were delivered in: the last version by position, or nothing after a deletion. That much is a last-writer-wins register and is order-free.

Two derived fields are weaker, and this paper states the weakness rather than claim past it. The creation time is kept as the least time over the id's records, and the two sticky asset fields are carried forward from a later record that omits them. The fold computes both only from the records it has applied: the least is lowered on each applied write, and a sticky field survives only across an applied version. So where a version is redelivered out of log order, or where a retraction and a later rewrite of the same id are delivered out of order, the live creation time or a sticky field can settle at a value the strict log-order fold would not reach. Under the least-time rule a live creation time cannot settle later than the log-order value (section 6.7). The other direction, a live creation time earlier than the log-order value, is what the rule leaves: a value lowered by a record the live projector applied and a later in-place refold cannot raise. Section 6.7 reports the residual, and section 11 states it as a boundary. The payload converges unconditionally; these two fields converge up to that residual. ■

Lemma 2 (two streams). Take a message a from one stream and b from another. Each effect reads a set of rows and writes a set of rows. Suppose no row a writes is read or written by b, and no row b writes is read or written by a. Then the node, edge and tombstone tables are the same after a then b as after b then a. The only effect that reads a row another stream owns is an edge write, which reads the node tombstones, because an edge is refused while either of its endpoints carries a tombstone (section 6.5). So the condition fails only for a pair whose edge names an endpoint the other message retracts, and it is not enough to ask only that the two write disjoint rows. ■

A derived row reads rows from several streams, so Lemma 2 does not cover it. It is recomputed from the current tables each time any of its inputs arrives, so once every input has landed and one later recompute has run, its value is a function of the final inputs. Two caveats bound that, and the paper states them rather than assert a clean convergence. First, the recompute is not serialised with the input writes: each stream's consumer dispatches on its own, so a recompute can read the tables between two inputs on different streams, and only a recompute

that runs after the last input gives the final value. In steady state the recompute that each input triggers supplies that last pass; under a burst the value is eventually correct once the inputs stop and one recompute follows. Second, a recompute whose write fails is logged and the message is acknowledged, not retried, so a persistent failure can leave a derived row stale until its next input. The replay tool rebuilds the node, edge and tombstone tables and compares those; it does not rebuild the derived tables, so replay parity (section 6.7) does not cover them. The derived rows are a read-through cache of the three authoritative tables, and the claim for them is eventual, not the per-id log-order equality of Lemma 1.

Theorem. Two projections that consume the same streams converge to the same tables, whatever the interleaving of the streams and whatever the delivery order within each, with the single residual Lemma 1 names in the creation time and the sticky fields. The proof swaps adjacent messages of different streams, using Lemma 2, until both interleavings are the same sequence; Lemma 1 fixes the result within each stream. Two kinds of pair fail Lemma 2′s condition. The first is a retraction and an edge to the retracted node: section 6.5 shows that this pair leaves the same tables in both orders, through the tombstone. The second is a legacy id written under two families on two streams, which write the same row; Lemma 1 orders those by the immutable store time, then the stream name, so they too converge. The derived rows follow from the final node and edge tables. ■

An edge before its node. The `edges` table has no foreign key to `nodes`. An edge that arrives before either endpoint is written as it is, and its tenant comes from its own subject. When the node arrives, it triggers the recomputation of any derived row. So this ordering satisfies the theorem.

6.5. The two order dependencies, and how the fold removes them

The one effect across streams. A retracted node removes the edges that touch it, and those edges belong to the edge stream. The projector consumes each stream on its own, so an edge can arrive after the retraction that should have removed it. An unguarded fold inserts that edge after the deletion and keeps it, and the live model and a rebuild disagree. No other effect in the projector writes or deletes a row owned by another stream.

The retraction therefore writes a tombstone for the node, and an edge is never written while either endpoint has a tombstone. Retraction then edge: the edge is refused. Edge then retraction: the retraction removes it. Both orders leave the same tables, so Lemma 2′s condition holds for this pair and the read model does not depend on how far each consumer has read. The edge is refused rather than written as deleted, so that it is never in the edge table and every existing read stays correct as written. The retraction and every edge write take an advisory lock on the node ids they touch, so the two cannot interleave inside the database either. A node's tombstone is never lifted: a node written again under a retracted id gets its row back, but no edge can touch it again, because stream sequence numbers are local to each stream and nothing on the edge stream can say whether an edge came before the node's retraction or after its return. That is the cost of the tombstone, and section 11 states it as a boundary. The alternative, one sequential consumer per tenant in the broker's arrival order, gives up the parallel consumers for every tenant to settle one pair.

The one dependency on delivery order inside a stream. When an upsert fails, the projector negatively acknowledges the message, and JetStream redelivers it later, behind newer versions of the same id. An unguarded fold lets the older version overwrite the newer. So every row carries the position of the message that wrote it, and a write applies only when its position is newer (section 6.4). A deletion's tombstone carries its position too, so an older delivery cannot bring the row back. Each row is a last-writer-wins register ordered by log position, a standard convergent replicated type [8], and delivery order stops mattering. A redelivered older version changes nothing.

The fold carries a version. A projector whose fold is newer than the read model's folds the whole log again at its start, in place, so a change to the rules reaches rows folded before it. A backlog is folded the same way: a stream whose consumer has 10,000 or more records left to deliver when the projector starts is folded only, with the derived tables recomputed once at the end, and the fold version does not move. The live path spends about a second per record on the derived rows, measured in the deployment, so a backlog of 1,464,604 records would take days on it. That figure is the one measure of projector throughput this paper has. It does not measure how long a write takes to become visible in the read model, nor how long an in-place refold takes or what a reader

sees during one. Read-model lag is a property of the deployment's load, not of the check, and is left unmeasured here. With the position guard and the tombstone, the theorem holds for every interleaving and every redelivery.

6.6. Shared drafts

People edit shared drafts in a Yjs document [9]. The draft is never written to the log and never replayed. An authority with a server sequence and last-writer-wins per slot settles accepted additions into the log. It validates them against the registry and writes them under ids derived from tenant, object and content. A settle repeated after a dropped connection writes the same id, so the log holds one record however many times it is sent. The draft layer converges by its own CRDT rules. The log converges by section 6.1. The settle step is idempotent, and nothing replays the draft.

6.7. The empirical check

Replay parity is the test of the fold. The replay tool reads every stream the projector binds from its first message, in log time across streams, and runs each record through the projector's own fold at its log position, into a fresh schema in a database of its own, so the rebuild writes nothing to the live server. It then compares every row of the node, edge and tombstone tables, rebuilt against live, by id and a hash of the columns the fold writes from the record. The live side is read by the reader role under the row policies, one tenant at a time, so the comparison cannot write. The report gives, per stream, the messages and the drops, and per table the rows only live, only rebuilt and differing, with the live-only edges by cause. It runs as a task inside the environment and records its run on the ledger as a job with one check per table.

I ran the replay inside the deployment on 19 October 2026, after the projector had folded the whole log again at fold version 3. It read 1,451,184 messages from 33 streams, the fact and presence streams among them, of which 22 are non-empty record streams, and no record was dropped; the run's five drops were worker presence facts, which are not records. Reading took 147 seconds, applying 393, comparing 12, and the whole run 551, each rounded to the second. The column written after the read counts live rows written, or deleted, by messages past the replay's read heads; those rows are left out of the comparison. So the rebuilt count is the live count less the only-live rows and the rows written after the read, plus the rows the live model deleted after the read.

Fold 3, 19 October 2026.

Table	Rebuilt	Live	Written after the read	Only live	Contents differ
Nodes	292,792	293,019	39	196	146
Edges	334,016	334,229	19	194	6
Tombstones	148	148	0	0	0

No row exists only in the rebuild, and every tombstone matches. The report classes all 196 live-only nodes and 194 live-only edges as carrying no log position; they are tags and tagged edges written on 10 September. Each differing row, all 146 nodes and 6 edges, differs in exactly one column, the creation time. In every one the live value is later than the rebuilt one, and the rebuild took its earlier value from a record the retained log still holds. Fold 3 keeps the creation time a row was first written with, so where the live projector met an older record after a newer one, a refold in place cannot lower the value, because the position guard skips an older record whole. Under that rule the value depends on the row's delivery history, not on the log. The rule of section 6.4, the earliest creation time of all the id's records, removes that dependence, and the run of fold 4 below measures it. A projector whose fold is newer than the read model's folds the whole log again in place at its start (section 6.5). The parity shown is containment of the rebuild in the live model, and the source of the live-only rows is not proved. The replay also applies the address check of section 2.2 and reports the records it refuses: a record off its address has no row in the rebuild. A readdress copies a tenant's records once to another tenant's root, each with its id and body, at its own address, so such a record lands where the projector folds it, and the rebuild has it. Each copy names the message it came from, so a second run copies nothing. Measured in the deployment, a readdress read 1,464,604 records from 24 streams, each to its head, and copied them in 12 minutes.

I ran the replay again inside the deployment on 20 October 2026, after the projector had folded the whole log again at fold version 4, the first with the rule of the earliest creation time. It read 1,454,979 messages, and no record was dropped. Reading took 117 seconds, applying 408, comparing 11, and the whole run 536. This run's report classes a live row with no message in the retained log as outside the retained log, apart from the rows found only live for any other cause.

Fold 4, 20 October 2026.

Table	Rebuilt	Live	Written after the read	Outside the retained log	Only live	Only rebuilt	Contents differ
Nodes	293,707	293,947	53	196	0	0	45
Edges	334,449	334,665	22	194	0	0	0
Tombstones	148	148	0	0	0	0	0

The columns do not sum to the live count: 293,707 plus 53 plus 196 is 293,956, nine more than 293,947, because nine of the 53 rows past the read heads were updates to rows that both models hold, and the tombstone table shows no deletion past the heads. Edges and tombstones match. The differences of the fold-3 run, in which the live creation time was later than the rebuilt one, do not occur: the earliest-creation-time rule does not admit that direction. The 45 nodes that differ now all differ in the creation time the other way, the live value earlier than the rebuilt one. That is the one direction the least-time rule leaves, and it has two possible causes, which this run did not separate per row. One is an earlier write the log does not hold, such as a write made before the log streams existed, so the rebuild never sees the earlier time. The other is the delivery-order residual of Lemma 1: a retraction and a later rewrite of the same id delivered out of log order, which can lower the live creation time below the log-order value while the earlier record is still retained. The report counts the 45 but does not test, for each, whether the earlier record is absent from the log. So I state these 45 as the residual the least-time rule leaves, by either cause, not as proof of either one. They, and the 196 nodes and 194 edges outside the retained log, are the boundaries section 11 names.

7. Related work

System	What it unifies	What it keeps apart	Where this system differs
Google Zanzibar [6]	Every access rule is a relation tuple `object#relation@user`. Group nesting is expressed by usersets in the same tuples. Namespace configurations with rewrite rules define the relations	Object contents stay in client storage. Configurations sit in their own table with their own changelog. Deep nesting needs the Leopard index	Here the rules are patterns on the name, not tuples per object, so no check chases pointers. The cost is that the rules are coarse: tenant, family and verb. Zanzibar is causally consistent through zookies; this system does not order a revoke against a write
Datomic [10]	Schema is data: attributes are installed by transactions in the same store as the facts, the datoms `[entity attribute value tx added]`	Access control is not part of the model	Here too the schema is in the log, as records on the platform root, versioned and append-only, and each process holds a local copy fed by subscription. What still differs: a change is a new identifier, never an edit, where Datomic can alter an attribute in a transaction; and Datomic has no tenant or grant in its key
EventStoreDB (KurrentDB) [38]	Streams whose names carry a category, access policies matched by stream-name prefix where the first match wins, and system projections that link each event into a category stream `$ce-<category>` and an event-type stream `$et-<type>`, both taken from names	The schema of an event is not part of the model, and a producer chooses the stream name	This is the closest event store: a type or category in the stream name, and access policy on the same name by prefix. What differs is that the type here is derived from an immutable schema identifier rather than chosen, the name has a fixed tenant and family depth, the registry is a family on the log, and an unknown type has no stream
The Linux virtual file system [11]	One inode carries type, permissions and position in the hierarchy	Hierarchy has any depth. Lookup checks execute permission on every	Here the depth is fixed at five tokens, so the walk is bounded by construction, not by a cache

System	What it unifies	What it keeps apart	Where this system differs
		directory it walks, made fast by the dentry cache and RCU-walk	
SpiceDB [12] and OpenFGA [13]	Open implementations of the Zanzibar model: a schema of definitions, relations and permissions, and relationship tuples	Application data and its schema stay outside. Checks recurse through relations; OpenFGA caps the depth with `resolveNodeLimit`, 25 by default	The same difference as Zanzibar. These systems answer "may this user see this object" for graphs of any depth. This system answers "may this identity write this family of this tenant" in bounded time, and leaves object-level rules to the one writer
Kafka with a schema registry and ACLs [14], [15], [16]	Topics, prefixed ACLs and a broker-side check, configurable per topic, that a record names a schema registered under the topic's subject	The registry is a separate service with its own store. A topic exists whether or not any schema maps to it. ACL names carry the topic, not the record type	The schema registry's strategies include a record-name strategy and a topic-record-name strategy, which derive a schema subject from the record's type, so the type-first direction is not new. Two things differ here. The address is the record's own, schema identifier to family to subject to stream, and both the broker's routing and the grant are written on that same name, so an identifier the map lacks has the family `unknown`, which has no stream. And the enforcement point differs: Confluent's broker-side validation refuses a wrong type at the broker, where this system's broker accepts any subject a grant allows and the projector is what refuses a record published off its address (section 3.4). So Confluent keeps a wrong type out of the log; this system keeps a wrong type out of the read model
Apache Pulsar [17], [18]	A schema per topic, stored with the topic. The broker checks a producer's schema against the topic's, and a namespace can require that every producer has one. Permission to produce is granted per namespace and checked per topic. Names are fixed depth, `tenant/namespace/topic`	The schema is attached to a topic that exists first. A topic name carries no type	Pulsar's names are the closest in shape to this system's fixed-depth tenant naming, and its schema enforcement and its grants both sit at the namespace. The difference is the direction and the unit: the schema is attached to a topic that exists first, not derived from the record, and an unknown type still has a topic. Here the identifier derives the address, and an unknown type has no stream
Google Pub/Sub [19], [20]	A schema associated with a topic. A message that does not conform is not published. Access is granted on the topic	The topic exists first, and the schema is attached to it	The same direction as Kafka and Pulsar: topic first, schema attached
Fauna's attribute-based access control [21]	Roles, membership predicates and collection privileges, stored in the same database as the data	Predicates are arbitrary functions over documents, evaluated per query. Fauna ended its service on 30 May 2025 [22]	Here the predicate is a pattern match on the name, with a bound, not a function with none
XACML 3.0 [23]	One policy language for subjects, resources, actions and environment, evaluated by a decision point	Data, schema and decision live in separate services: enforcement point, decision point, information point	Here the decision point is the broker's subject matcher, in the path of every message. There is no separate decision service to call
Capability systems [24], [25]	The right to act is a token the holder presents; delegation narrows it	No global name tree is required	A NATS grant is closer to an access list than to a capability: it is bound to an identity's key and held by the server. NATS's JWT mode moves toward capabilities, with signed user claims that a server verifies with no lookup

System	What it unifies	What it keeps apart	Where this system differs
NATS subjects, permissions and accounts [26], [27]	A stream binds the subjects it keeps, and no two streams may keep the same subject. Permissions are allow lists of subject patterns per identity. Each account is its own subject space, joined to others only by explicit imports and exports	Schemas are outside NATS. A registry for NATS exists as a separate key-value service [28]	Two of this system's relations, the stream topology and the grants, are stock NATS: streams bound to subjects, and permissions written on the same subjects. That half is not new and is named here plainly. What this system adds is the `Kind` function that derives the family from the schema identifier, the `unknown` family with no stream, the fixed five-token depth, and the schema registry as a family on the same log. It uses one account with the tenant as a token rather than an account per tenant
Hermes [29], [30], [31]	A hash of the event type name is the id of the type's rendezvous node, which holds that type's access policy, and each event is type-checked at publish against a type repository	The publisher chooses the type. There is no durable replicated log, no per-type durable storage topology, no tenant token and no read model; the name has no fixed depth and the broker walks subtypes	Hermes is the nearest prior system, and it is closer than a reader of the other rows would expect: the type name already computes a routing address and the placement of that type's access policy. What is new here is the rest of the invariant held together: the type computes a durable per-tenant stream, not a routing node; an unregistered type has no stream to land in; the depth is fixed at five tokens; and the registry is itself a family on the same log. The claim is the combination, not type-addressed policy alone, which Hermes already has
Named Data Networking's trust schemas [32]	A packet's hierarchical name and the keys that may sign it. A trust schema is a set of rules over name patterns, binding a data name to its signing key by shared prefix and by components at fixed positions, so whether an identity may publish a packet is a pattern match on the name, not a lookup in a separate access list. Name-based access control grants read access per namespace and dataset through named keys	NDN forwarding is stateful: routers keep the state of every pending interest. Names have any depth, and trust is a chain of signatures the consumer checks. There is no log	Here the name has five tokens and the broker checks it before the write. There is a replicated log, a stream per family, a schema registry inside that log, and a deterministic fold into a row-secured read model. NDN has none of them
Tuple spaces: Linda [33]	Routing and storage under one mechanism. A process writes a tuple into a shared space, and another reads or withdraws it by matching a template	No access control, of bounded depth or otherwise, and no schema registry: any process may match any tuple	Here the pattern that routes a record is also the rule that grants it and the binding that stores it, and the type of every record is registered in the log
CloudEvents [39]	A standard event envelope whose `type` attribute is defined for routing, observability and policy enforcement, and whose `dataschema` names the event's schema	A format, not a system: it defines attribute names, not a broker, a store or an enforcement point	CloudEvents already puts the type and a schema reference in a standard envelope and names policy enforcement as a use of the type. It is an envelope convention; this system is the broker, the store, the registry and the grant, all indexed by that name

What none of these does is make the type set the durable storage topology and write the access rules on the same names, with the type derived from an immutable schema identifier, a fixed tenant and family depth, an unregistered type that has no stream to land in, and the registry held as a family on the same log. That combination is the narrow form of the claim. Each part alone is old. Writing access rules on type names is in Hermes and EventStoreDB. Deriving a schema subject from the record's type is in Confluent's registry. Binding streams and permissions to the same subject names is stock NATS. A type in a standard envelope used for routing and policy is CloudEvents. The claim is that the four constraints of section 1 hold together as one invariant, not any one of them.

8. Micro-benchmarks

The full tables and the method are in the benchmark report [5]. Every number in the first three tables was measured on one Apple M5 Max (18 cores, macOS 26.5.1) on 19 October 2026. Docker is not installed on that machine, so Postgres 18.3 and Valkey 9.1.2 ran as native processes over loopback. The schema registry stand-in ran as its own process over HTTP. The broker was the nats-server library in the benchmark's process: one node, file storage, no wait for disk sync.

The evaluation of one write. A workload of 20 actions, 16 allowed and 4 refused. The comparison path asks Postgres for the stream, Valkey for the grant and the registry for the schema.

Path	Workers	p50	p99	Throughput
Local, one process	1	5.96 µs	23.62 µs	126,516 per second
Three services, in sequence	1	144.88 µs	184.96 µs	7,206 per second
Three services, registry in parallel	1	101.21 µs	179.04 µs	9,856 per second
Local, one process	18	10.96 µs	529.46 µs	558,484 per second
Three services, in sequence	18	469.88 µs	892.50 µs	37,888 per second

At eight workers and above, the local path's p99 is set by garbage collection of the validator's allocations, not by the lookups. With the collector off, the 18-worker p99 falls from 529 µs to 78 µs. The deep tail is worse for the local path than this table shows, and the benchmark report [5] carries it in full: at 18 workers the local p99.9 and max run above the services', and at 64 workers the local path's max reaches tens of milliseconds while the services' stays near five. Two things drive that: the collector's pauses, and that each local worker runs ten times as many operations as a service worker in these runs, so its tail is sampled far more deeply. The local path wins at the median and the throughput, not at the deep tail under heavy concurrency, and a reader weighing worst-case latency should take the p99.9 and max from the report, not the p50 here.

One write end to end. The checks, then a publish to the broker, waiting for the stream's acknowledgement.

Path	Workers	p50	p99
The checks, then publish (harness "publish only")	1	46.96 µs	77.88 µs
Local checks, then publish	1	45.67 µs	67.38 µs
Three services, then publish	1	188.54 µs	251.17 µs
Local checks, then publish	64	454.29 µs	1,461.33 µs
Three services, then publish	64	1,791.17 µs	2,866.83 µs

A caveat on the first row. The harness's "publish only, no checks" path runs the same local checks before publishing as the row beneath it, so the two rows measure the same work and their near-equal numbers say nothing on their own. The cost of the local checks is the evaluation of section 3.2, about 6 to 9 microseconds for this workload's mix, set beside a loopback publish of about 47 microseconds. So the checks are a small part of a single-node loopback publish, and a far smaller part of the deployed publish below. The three service calls add about 140 µs at one worker and about 1.3 ms at 64.

One read by id under the tenant policy. 100,000 rows across three tenants. One read in ten asks for another tenant's row and must come back empty; every one did.

Path	Workers	p50	p99
Plain read, no policy	1	26.04 µs	36.83 µs
Row policy, bound in three round trips	1	71.58 µs	91.00 µs
Row policy, pipelined (one round trip)	1	30.62 µs	42.96 µs
Valkey grant, then a filtered query	1	94.83 µs	113.58 µs
Row policy, bound in three round trips	64	1,296.88 µs	1,753.29 µs
Row policy, pipelined	64	436.42 µs	733.33 µs
Valkey grant, then a filtered query	64	828.25 µs	1,302.04 µs

The policy and its binding, sent in the same round trip, add about 4.6 µs to a plain read. The three-trip row opens a read-only transaction with the reader role and the tenant set, reads, and commits. At 64 workers it is slower than the Valkey path, because Postgres is the bottleneck and the binding triples the trips to it. Sent as one pipeline, the bound read costs what a plain read costs, and it beats the Valkey path at every concurrency measured. One caveat on that comparison: the Valkey path does a membership lookup (may this identity read this tenant) that the policy path does not, because in the services that membership check happens earlier, in the edge's token or the key lookup, and the policy read takes the tenant as already bound. So the two paths are not quite the same work; the policy read's advantage is partly that its tenant arrives bound rather than looked up per read.

The services bind as the pipelined row does. The binding and the read go as one pipeline with one sync, which Postgres runs as one implicit transaction. The binding statement sets the reader role, the tenant as a bound parameter, and the transaction's read-only flag, and the forced row policy and the refusal of an unbound read are unchanged. Measured with the services' own binding on the same machine, in alternating runs of the three-trip binding and the pipelined one:

Services' bound read	Workers	Mean	Throughput
Three round trips	1	87.90 µs	11,371 per second
One round trip	1	41.23 µs	24,232 per second
Three round trips	64	1,392.35 µs	45,870 per second
One round trip	64	446.61 µs	142,564 per second

In the one-round-trip run at 64 workers, the plain read took 453.84 µs and the Valkey grant then a filtered query 909.32 µs, so the services' bound read costs what a plain read costs and takes half the time of the grant-then-query path.

The same harness inside the deployment. I ran the harness again on 19 October 2026 as a task in the cloud region that holds the cluster, on two cores of an Intel Xeon Platinum 8259CL with Go 1.25.14. Postgres 16.15 was the deployment's own server, reached over the network, with a scratch database dropped at the end. No Valkey is reachable there, so the comparison path kept its grants in a Postgres table in the same database as the stream table. The registry stand-in was served over HTTP on loopback by the benchmark's own binary. The broker was the three-node cluster, nats-server 2.11.0, and the stream had three replicas on file storage, so every acknowledgement waited for a majority. Every row below had zero wrong answers.

Path	Workers	p50	p99	Throughput
Local, one process	1	18.88 µs	141.15 µs	36,430 per second
Three services, in sequence	1	791.02 µs	992.37 µs	1,344 per second
Local, one process	64	36.74 µs	198.51 µs	38,179 per second
Three services, in sequence	64	7,719.68 µs	27,796.23 µs	7,050 per second

Path	Workers	p50	p99
The checks, then publish (harness "publish only")	1	6,069.23 µs	9,319.71 µs
Local checks, then publish	1	6,042.69 µs	15,563.32 µs
Three services, then publish	1	6,881.92 µs	8,502.43 µs
The checks, then publish (harness "publish only")	64	9,032.55 µs	22,855.97 µs
Local checks, then publish	64	8,749.30 µs	40,204.56 µs
Three services, then publish	64	13,832.98 µs	28,821.18 µs

The publish latency includes the task's network path to the cluster, so the figure is the path as deployed, not the broker's consensus alone. As on the laptop, the harness's "publish only" row here also runs the checks, so it is not a no-checks baseline; the point the two rows do make is that the checks' 6 to 9 microseconds are lost in a 6 millisecond deployed publish (6,043 µs against 6,069 µs at p50). That is what zero-hop verification (section 3.5) predicts: against a network publish the local checks cost nothing that shows. The three calls add about 0.8 ms

at one worker and about 4.8 ms at 64. At one worker each path ran 500 operations, so its p99 is the fifth-slowest write, and the p50 is the comparison. The local path's p99 sits above the three-service path's. The cause was not measured; my reading is that the sequential service calls pace the writes arriving at the stream's leader, where the local path sends them back to back.

Path	Workers	p50	p99
Plain read, no policy	1	255.11 µs	375.88 µs
Row policy, bound in three round trips	1	704.89 µs	1,007.91 µs
Row policy, pipelined (one round trip)	1	303.21 µs	511.80 µs
Postgres grant, then a filtered query	1	459.13 µs	911.65 µs
Plain read, no policy	64	3,942.76 µs	12,536.24 µs
Row policy, bound in three round trips	64	11,864.16 µs	21,230.18 µs
Row policy, pipelined	64	6,609.32 µs	20,502.15 µs
Postgres grant, then a filtered query	64	7,731.39 µs	16,541.75 µs

Over the network the round trips dominate. The three-trip binding adds about 450 µs to a plain read at one worker, and at 64 workers it is the slowest path. The pipelined binding, the services' path, adds about 48 µs at one worker and is faster than the grant-then-query path at both concurrencies. At 64 workers on two cores every path queues at Postgres, and the pipelined read takes 1.7 times the plain read.

What these numbers are not.

- The laptop's numbers are loopback numbers. The run inside the deployment shows what each round trip costs over a network: the gap between the paths widens.
- The comparison path calls every service on every operation, and the local path does not, because its registry and its grants are local: the registry is a map fed by subscription, which section 2.3 calls a cold-start cache, and the grants are held in the broker that the write already goes to. So this is a cached-local path against an uncached-remote one. A production comparison stack would cache grants and schemas in the writer too, and most of the latency gap would close. What the local path keeps is not raw speed over a cache but structure: the cache is fed by the same log the records are on, and the check is in the process that holds the record, so there is no separate service to call and no second invalidation path. A cache of a separate service still has its own staleness window; so, honestly, does the local registry, between a schema's append and a reader's subscription catching it, which section 2.3′s bounded re-read closes.
- On the laptop the broker had one node and did not wait for disk sync. Inside the deployment it has three, and a write waits for a majority.

9. Toward one physical store

The three structures can be made one store. This section states that alternative with its costs. It is not part of the claim.

The shape. One replicated table, keyed by subject pattern:

```
(pattern, schema_id?, schema_hash?, stream?, identity?, verb?, tenant)
```

The natural home is a JetStream key-value bucket, which is itself a stream, so the table sits in the same log as the records. Every writer and every broker holds a local copy and watches the bucket for changes.

What it needs.

- The broker reads grants from the table. NATS can delegate the decision at connect time to an external service, called auth callout [34]. Permissions can also be issued as signed user JWTs [27]. Either way, a grant change reaches a server as data, with no change to its configuration file. The callout is consulted when a connection is established, so a grant changed as data still has to force live connections to reconnect or be dropped for the change to bind, exactly as a configuration reload does; the win is that the change is data, not that it is instant.
- Writers resolve schemas and stream bindings from the same table that holds the grants.

- Every evaluation would need to name the table version it used, so that a write and the grant it passed under can be ordered, the role Zanzibar's zookies play. This is the hard part and the broker gives no mechanism for it: the broker stamps nothing on a message, and a version a writer stamps on its own record can be forged. Ordering a write against a grant change would need either a broker change or the authority as the single writer that stamps the version it read. The one-store sketch does not solve this; it inherits the same gap section 5 and the "revoke per layer" boundary already name.

What it costs.

- Schemas become mutable at run time. The protection of section 6.2, that a conflicting variant cannot reach the log, is lost. It must be rebuilt as a rule in the table: append-only versions, never edits.
- Every reader holds a cache of the table, so every reader can be stale. A revoke is then as fast as the slowest cache.
- Structural changes and data writes share one log again. They do not share a lock, because the bucket is its own stream. They do share the broker's capacity and its failure modes.

The middle path is the one taken. The tenant and user records on the platform root are the source, and each enforcement point takes its own form from them: the bootstrap emits the broker's grants in the broker's configuration shape and a configuration change carries them to the servers, the credential carries its tenant and memberships, and the row policy compares the transaction's tenant with the row's. That keeps one source without one store. Each enforcement point keeps its local, bounded check.

10. Deployment at the edge

A site that loses its connection keeps working and catches up when the line returns. A store that loses its connection keeps taking orders and reconciling stock, and the records reach the hub when the line returns.

10.1. The hub

The hub is a three-node JetStream cluster with JetStream in a domain of its own, and a leaf listener on the private network. A writer with no site dials the hub directly, as a client: it discovers the hub's brokers over the private network and holds a connection to each. Each broker advertises its private-network address on the leaf port, and the brokers pass each other's on, so a site's leaf learns every broker by an address it can dial and follows a roll of the cluster by itself.

10.2. The site

A site runs its own NATS server as a leaf node of the hub [35], with JetStream in the site's own domain, so the site's streams and the hub's never claim the same name [36]. The server listens on loopback only; nothing off the box dials it. Its one outbound connection is the leaf link, which redials the hub every second while the line is down.

A site is named by a spec of three things: the families it writes, the hub streams it reads, and the hub's domain. For each family it writes, the site holds a stream with the hub stream's name and the hub stream's subject, `tenant.<tenant>.dag.<family>.>`, on one replica, on file storage. A record published at the site is captured and acknowledged by the site's stream whether or not the hub is reachable. Each site stream is totally ordered by the site's server, by the argument of section 6.1 applied to one server. Each hub stream the site reads is mirrored at the site under its own name. The bootstrap that creates the site's streams and mirrors never updates or deletes one: a stream that exists with another configuration is reported and left as it is.

The hub's stream of each family the site writes takes one source from the site's stream of the same name [37]. A source copies messages from another stream, across the leaf link, from where it left off. The site's records therefore arrive at the hub in the site's own order, after the link returns, and the site's stream stays the site's. The source is filtered to its own family's subjects under the site's tenant root, so a site stream can deliver into a hub stream only records of that stream's family and tenant, and the projector's check that a record's family is its stream's family refuses anything else. The leaf refuses the site's family subjects in both directions. A record published at the site reaches the hub only by the source, so the hub's stream holds it once. A record published at the hub is never captured at the site, so it is never sourced back.

The grant. The leaf connects to the hub as the site worker's own identity, so the site holds no credential of its own. A NATS grant belongs to an identity, not to a kind of connection, so the leaf's grant is the worker's, and it includes the worker's two record families. What keeps plain records off the link is the leaf's own deny lists: the site's family subjects are refused in both directions, so a record reaches the hub only by the hub's source. The hub does not take a site's word for a record's family: the source filter and the projector's family check confine what a site delivers to the families it was given. A site is still trusted for the validation of what it writes within those families, as any family writer is (section 11). Beside the families, the grant names the worker's presence and fact subjects, its work queues through the hub's domain, each consumer create pinned to the worker's own filter, and the JetStream subjects that carry the mirrors, the sources and their acknowledgements. A site holds one tenant's families: the tenant whose root carries the site worker's queues, facts and presence. The spec refuses any other.

Work and facts. The work queues stay at the hub. The site's worker binds them through the hub's domain, across the leaf link, and its presence and facts cross the link as plain messages that the hub's streams capture. While the line is down, the site writes its own records and takes no new work. Records written at the hub from facts reach the site through its mirrors.

Convergence. A hub stream that sources from a site holds the hub's own writes and the site's records, interleaved in arrival order. It is still one stream, so Lemma 1 covers it: each message has one position in the hub's stream. The deterministic fold carries over unchanged, and so do the position guard and the tombstone of section 6.5. Deterministic ids (section 2.1) make a record published twice land as one row.

In the deployment. Two sites run as leaves of the hub, each with its own JetStream domain, its own asset and edge streams and a mirror of the post stream, and the hub's asset and edge streams source from both. The disconnect figures of section 10.3 are from the laptop test.

10.3. Costs and limits

- **The hub-to-site grant surface.** Each site adds a leaf grant at the hub and one source on each hub stream of the families it writes. A second site is a second domain, with its own source on each of those streams; when two sites share one identity, its grant is the union of the two sites' grants. The grant list and the stream declarations grow with the number of sites.
- **Delivery subjects carry no stream.** The subjects that carry a source's deliveries hold random ids that a grant cannot narrow by stream, so a site is trusted not to publish on deliveries that are not its own.
- **Stream storage at the site.** A site keeps its own streams on its own disk, with a file store of 8 GB. A long disconnection fills it. A publish refused for space is an error at the writer, which leaves the work for a later pass.
- **The ordering interleave.** A hub stream sourced from several sites is ordered by arrival, not by the sites' clocks. Two sites that write the same id, which deterministic ids allow only for the same record, converge by Lemma 1. Two sites that write related records converge under the theorem's condition, and break only where it breaks.

Disconnect and reconnect. A test runs a hub and one or two sites in one process on a laptop, nats-server 2.14.0, each site started from the shipped configuration and joined to the hub by a line the test cuts and restores. The hub advertises a closed port, so no leaf can dial around a cut, and the test checks that every leaf is down before the lines return. The hub writes 25 assets, and the lines go down. Each site writes records of its own two families, half assets and half derivation edges, acknowledged in its own domain, while the hub writes 25 more. The lines return, and the test reads the hub's streams back. With two sites, each has its own domain and its own source on each of the hub's streams, and one identity holds the union of the two grants. Three runs each:

Sites	Records per site	Site writes, each site	Link up after the line returns	All at the hub after the line returns
1	2,000	84 to 93 ms	2.0 to 2.7 s	2.2 to 2.8 s
2	2,000	82 to 85 ms	2.0 to 2.6 s	2.2 to 4.4 s
1	20,000	1.2 to 2.0 s	0.8 to 1.2 s	0.9 to 1.4 s

Sites	Records per site	Site writes, each site	Link up after the line returns	All at the hub after the line returns
2	20,000	0.9 to 1.4 s	0.6 to 2.8 s	2.7 to 3.1 s

In every run the hub held every record of every site exactly once, each site's records in that site's order, through one source per site, at hub positions after the hub's own writes. Nothing was refused at the hub or at a leaf. In the harness the leaf alternates its one-second redials with the closed port, so the link returns within about three seconds of the line. Once the link is up, sourcing takes 0.14 to 2.2 seconds, and the spread does not follow the record count: ten times the records arrive in the same window, so the time is the sources re-establishing, not the transfer.

Read-model parity at the site. A second test runs a projector at each end after a disconnection. The site writes its assets and the derivation edges between them; the hub writes 50 posts that cut from those assets, ten of the site's assets byte for byte, as the cloud worker does when it takes over a task, and one asset the site never sees. When the line returns, one projector folds the hub's streams and the other folds the site's streams and its mirror. Three runs at each size:

Records written at the site	Line down	All sourced after the line returns	Read models settled after sourcing
2,000	5 s	213 to 1,103 ms	1.4 to 1.6 s
20,000	5 s	551 to 2,650 ms	100 to 104 s
20,000	60 s	2,030 to 5,317 ms	53 s

In every run the two read models were equal in all five comparisons: the asset nodes, the edges, the two derived tables of assets, and the mirrored posts. The hub's own asset never reached the site, and nothing crossed the link as a plain message. Each asset written at both ends was one message at the hub and one row: within its duplicate window, the hub's stream drops the sourced copy of a message id it already holds. Beyond the stream's duplicate window, the hub stores both copies and Lemma 1 folds them to the same row. The settle time is two projectors folding into one Postgres on one laptop, not the link.

10.4. Uses beyond the studio

The deployment this paper measures carries one kind of tenant, a creator's studio pipeline [1]. Its numbers are the ones in sections 6.7, 8 and 10.3, and nothing below adds to them. The primitive is not specific to footage: a tenant's record names its kind, a schema record names the kinds that carry its family, and the bootstrap creates the streams (section 5). Three other uses follow. Each is a design on the primitive as built, not a deployment, and each is stated with what supports it.

Venues and timed entry. A companion paper [40] specifies a venue tenant: a complex at the scale of a theme park, with timed windows on rides, screenings and tables. A window is one timed slot of one space, and a place is a key from the seat map, or a number up to the window's capacity where seats are not assigned. A reservation's id is derived over the tenant, the family, the window, the place and an epoch, so one place in one window has one subject on one stream. Its first version is published expecting that subject to be empty, and the stream's leader stores one such write and refuses every other. That is how a deterministic id prevents a double sale, and the companion proves it with the conditions it needs of the broker. Holds and reservations are written only at the hub; a site in the park admits tickets and sells from an allotment the hub gave it, while cut off. The venue has no tenant record and no schema record on the log, and every measured number in that paper is this one's.

Edge sensors and telemetry. A site of section 10.2 ingests locally. Devices reach the site's worker over the site's own network, the worker writes records on the site's streams, and the hub sources them when the line returns. That is the shape section 10.3 measured, with one or two sites. The read model holds state: one row per id, its latest version by log position (section 6.4). It is not built to hold a series of samples, which a time-series or streaming engine holds better. No tenant of this kind exists, no family for it is specified, and the consumer that would carry samples off the log is the subject of section 12.

Disconnected retail and inventory. The studio paper [1] specifies a store kind: products, purchase orders, shipments, orders, fulfilments and stock moves, among others, where stock on hand is the sum of the stock moves, and a connector that publishes a storefront's orders as facts. None of these families is in the registry, and the deployment runs the creator only. In the design, a store's site writes its orders and stock moves on its own streams and keeps taking orders while the line is down, and the hub's streams source them when it returns, as in section 10.2. Deterministic ids make an order reported twice, once by the site and once by the storefront's cloud, one row (Lemma 1). What the primitive does not do is stop two sites that are both cut off from selling the same last unit, since each site's stream orders only its own writes. A store needs a bound of its own for that, such as the venue's door allotment: a share of places the hub gives a site before the line goes down, which the site may sell and no other [40].

11. Boundaries of the claim

The trust model. The primitive's guarantees hold against a stated set of trusted parties, and this paragraph names them so a reader knows what is assumed rather than proved. Trusted: the broker and its Raft consensus, under the crash model of section 6.1; the bootstrap identity, which adds every family, tenant and schema and administers streams; the emergency identity, which holds everything; each family writer, within its family; a site's own server, for the validation of the records it writes in its own families (section 10.2); and each read-model service, which binds the tenant of a read from the caller's credential (section 3.4). A holder of a credential for one tenant is confined to that tenant by the grant table at the broker, by the address check in the projector and by the row policy in the read model. Within its tenant, a grant on a record family is coarse by design (section 4): any identity granted a family may overwrite or retract any record of that family, and fine-grained, per-record decisions are the authority's, made before it writes. The log does not record which identity published a record, so the audit answers who could write, from the grant history, not who did. The keys of the two privileged identities are the system's highest-value secrets; their storage, use and rotation are an operational matter this paper does not cover.

Six properties bound what the primitive promises. Each is a consequence of the construction, stated here with what it costs. Three further limits, on scale, on deletion and on recovery, follow them.

A family writer is trusted within its family. A grant names a tenant and a family, not records, and the log carries no authenticated publisher. The consequence is that a compromised family writer can rewrite or retract its family's records within its tenant, and nothing in the log says which identity did it. It cannot reach another family or another tenant: the grant table refuses the subject at the broker, and the address check refuses any record whose subject is not its own five-token address on its own family's stream. The property is what keeps the check a bounded match on the name (section 4); narrowing it needs per-record rules, which belong to the authority before it writes.

One naming, not one store. The registry, the tree, the grants and the row policy share one key and live in four places (section 2.6). The consequence is that a change that spans them is three changes by three mechanisms, never one write: a family is added by a schema record and by the streams the bootstrap creates for it, a tenant by a tenant record and the streams the bootstrap creates under its root, and a grant by a configuration change, not by any record alone. The property is what gives section 5 its no-shared-lock result. Section 9 gives the one-store alternative and what it costs.

Access along edges is not a grant. The check reads the five tokens of the name and nothing else (section 4). The consequence is that a rule which follows lineage, such as "a reader of this record may read what it was derived from", cannot be written as a grant and lives in the reader: in the authority before it writes, or in a closure index kept beside the log. The property is what makes the check's cost independent of the depth of any graph in the data.

A revoke is ordered per layer, not across layers. Each layer closes its own window at its own moment: the broker at each server's reload, the read model at the next transaction, and never against a write already accepted (section 5). The consequence is a window between the edit and the last layer's close, in which a write that passed

the old grant stands in the log. Ordering a revoke against a write needs a version on every evaluation, which is the role zookies play in Zanzibar and the cost section 9 names.

A retracted id takes no edge again. A node's retraction leaves a tombstone that is never lifted (section 6.5). Stream sequence numbers are local to each stream, so nothing on the edge stream can say whether an edge came before the node's retraction or after its return. The consequence is that a node written again under a retracted id has its row and no edges, and a thing that must come back with edges comes back under a new id. The property is what lets each stream keep its own consumer while the read model converges; lifting it needs one order across the node and edge streams, which is one sequential consumer per tenant.

Replay parity is containment, by id. Replay parity is containment against the retained log: every row the rebuild produces is present in the live model by id, and the live rows the rebuild lacks are not explained by the log (section 6.7). The match is by id and by the columns the fold writes from the record; the creation-time residual of Lemma 1 is the one column the two runs reported differing on, in 146 rows at fold 3 and 45 at fold 4. A record stream never drops a record (section 2.2), so the live rows the rebuild cannot reproduce are the rows whose records predate the log streams: the 196 nodes and 194 edges of fold 4 carry no log position and were written on 10 September, before the streams that hold the record families. The consequence is that a rebuild proves what the log implies, by id, not where every live row came from or what every column once held. The property is what lets the read model be dropped and rebuilt from the log alone, with those pre-log rows as the stated remainder.

Cost grows per tenant and per family. Every tenant adds one stream per family of its kind, each a Raft group of three replicas with its storage cap reserved in full on every replica, and the projector holds one durable consumer per stream. So the reserved storage, the number of Raft groups and the number of consumers all grow with the number of tenants times the families each carries, and the authority's grant grows with the number of tenant roots. All tenants share one NATS account, so the account's limits on streams and consumers are shared across them. The reference deployment runs comfortably at its tenant count, but a deployment with many tenants or many families is bounded by the cluster's capacity for replicated streams, not by anything in the per-write check, and its caps must be sized against the account's limits. Adding a tenant or a family blocks no existing stream (section 5); it does consume cluster capacity.

Deletion is not erasure across copies. A retraction removes a node and its edges from the read model and leaves a tombstone, but the records stay in the log, which never drops them, and the log is replicated three ways, mirrored at each site that reads it, and may have been copied once by a readdress (section 6.7). The primitive gives no single operation that erases one subject's data across all of those copies. A deployment with a legal erasure duty must build that erasure as its own procedure over the streams, the replicas, the mirrors and any readdressed copy; the primitive does not provide it, and because a record stream denies delete and purge, that procedure begins with a deliberate change to the stream's configuration.

Recovery and schema withdrawal are operational. Two limits are named here so a reader does not assume them solved. Losing a majority of a stream's replicas permanently is beyond the crash model of section 6.1; the log is the only source the read model rebuilds from, and backup and restore of the log are an operational matter this paper does not specify. And a schema identifier, once appended, cannot be withdrawn: a wrong or overly permissive schema validates its records forever, including on replay, and the remedy is a new identifier plus ceasing to write the old one, not a deletion. Neither is a property of the primitive; both are work a deployment must carry.

12. Next directions: large-scale distributed verification and downstream streaming

The measurements in this paper come from one laptop, and from one deployment of a three-node hub with two sites and a read model whose node and edge tables held about 629,000 rows on 20 October 2026 (section 6.7). Four pieces of work would test the claim at larger scale. None of them is done, and no result of theirs is claimed here.

The read model at 50 million to 250 million rows and beyond. The benchmark reads by id from 100,000 rows (section 8). At fifty to two hundred and fifty million rows, the measures are: the latency of the pipelined,

policy-bound read by id as the primary-key index grows deeper; whether partitioning the tables by tenant lets Postgres prune every partition but the caller's, so that a read touches only its own tenant's index; and the projector's rate with its upserts batched, set against the second per record the live path spends on derived rows (section 6.5).

Multi-node and WAN partition stress. The deployed latencies come from one task in the cluster's own region, and the disconnection tests ran a hub and one or two sites in one process (section 10.3). The next measures are the p99 and p99.9 of writes and reads across the three-node hub under injected loss, delay and jitter; the time to elect a new stream leader under those conditions, and what a writer sees meanwhile; and many sites reconnecting at once after a shared outage, with the time their sources take to catch up and the load they put on the hub's brokers. The venue's park, with a site per cart [40], is the case that needs the last.

A reproducible package. The deployment is declared in infrastructure definitions written for one cloud account, and the benchmark is public [5]. A package another reader can run would provision the hub, a set of simulated leaf sites and a Postgres server on local NVMe storage from declarations, and one self-contained chart would deploy it, seed it, run the benchmark of section 8 and the stress tests above, and report. A reader could then reproduce the deployed numbers, not only the laptop's.

Downstream analytical and time-series consumers. A consumer of the log needs a subscribe grant on `tenant.<tenant>.dag.<family>.>` and a durable consumer pinned to that filter (section 3.3). An analytical job or a time-series store could read the families it needs that way, from the streams, with no load on the read model. It would read the log as published, so the schema guarantee it gets is the writer's, not the projector's (section 3.4), and the grant that confines it is the same match on the name as every other. Building one, and measuring what it costs the brokers, is the next step for the telemetry use of section 10.4.

References

1. MOG Systems, Inc. *One Table: A Studio Footage Pipeline Run by One Person.* The companion paper. https://mog.systems/papers/one-table/ONE_TABLE.pdf
2. NATS. *JetStream.* https://docs.nats.io/nats-concepts/jetstream
3. JSON Schema. *JSON Schema: A Media Type for Describing JSON Documents*, draft 2020-12. https://json-schema.org/draft/2020-12/json-schema-core
4. nats-io. *nats-server*, v2.14.0, the version read. The subject trie is `server/sublist.go` and `server/gsl/gsl.go`; the per-connection permission cache and the subscription pruning on reload are in `server/client.go` and `server/reload.go`. The cluster runs 2.11.0, which has the same `reloadAuthorization`, `processSubsOnConfigReload` and 128-subject cache. https://github.com/nats-io/nats-server/tree/v2.14.0
5. MOG Systems, Inc. *primbench: the benchmark.* https://github.com/MOG-Systems-Inc/primbench. Its report, `BENCH.md`, is published beside this paper.
6. R. Pang, R. Cáceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, M. Wang. *Zanzibar: Google's Consistent, Global Authorization System.* USENIX ATC 2019. https://www.usenix.org/conference/atc19/presentation/pang
7. NATS. *Configuring NATS Server*, configuration reload. https://docs.nats.io/running-a-nats-service/configuration
8. M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. *A comprehensive study of Convergent and Commutative Replicated Data Types.* INRIA RR-7506, 2011. https://inria.hal.science/inria-00555588
9. Yjs. *Yjs: a CRDT framework.* https://docs.yjs.dev
10. Datomic. *Schema reference.* https://docs.datomic.com/schema/schema-reference.html
11. The Linux kernel. *Pathname lookup.* https://www.kernel.org/doc/html/latest/filesystems/path-lookup.html
12. AuthZed. *SpiceDB schema.* https://authzed.com/docs/spicedb/concepts/schema
13. OpenFGA. *Configuration.* https://openfga.dev/docs/getting-started/setup-openfga/configuration
14. Confluent. *Schema Registry.* https://docs.confluent.io/platform/current/schema-registry/index.html
15. Confluent. *Broker-side schema ID validation.* https://docs.confluent.io/platform/8.0/schema-registry/schema-validation.html
16. Apache Kafka. *Authorization and ACLs.* https://kafka.apache.org/documentation/#security_authz

17. Apache Pulsar. *Understand schema.* https://pulsar.apache.org/docs/next/schema-understand/
18. Apache Pulsar. *Authorization and ACLs.* https://pulsar.apache.org/docs/next/security-authorization/
19. Google Cloud. *Pub/Sub schemas.* https://docs.cloud.google.com/pubsub/docs/schemas
20. Google Cloud. *Access control with Identity and Access Management*, Pub/Sub. https://docs.cloud.google.com/pubsub/docs/access-control
21. Fauna. *Roles.* https://docs.fauna.com/fauna/current/learn/security/roles
22. The Register. *Fauna to shut down its database service.* 24 March 2025. https://www.theregister.com/2025/03/24/faunadb_shut_down/
23. OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0.* https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html
24. J. B. Dennis, E. C. Van Horn. *Programming Semantics for Multiprogrammed Computations.* Communications of the ACM 9(3), 1966. https://doi.org/10.1145/365230.365252
25. A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, M. Lentczner. *Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.* NDSS 2014. https://www.ndss-symposium.org/wp-content/uploads/2017/09/04_3_1.pdf
26. NATS. *Accounts.* https://docs.nats.io/running-a-nats-service/configuration/securing_nats/accounts
27. NATS. *JWT authentication.* https://docs.nats.io/running-a-nats-service/configuration/securing_nats/jwt
28. Synadia. *Schema registry architecture.* https://docs.synadia.com/platform/schema-registry/architecture
29. P. R. Pietzuch, J. M. Bacon. *Hermes: A Distributed Event-Based Middleware Architecture.* 1st International Workshop on Distributed Event-Based Systems (DEBS), Vienna, 2002, pp. 611-618. https://www.cl.cam.ac.uk/research/srg/opera/publications/papers/prp-debs2002-hermes.pdf
30. A. Belokosztolszki, D. M. Eyers, P. R. Pietzuch, J. Bacon, K. Moody. *Role-Based Access Control for Publish/Subscribe Middleware Architectures.* International Workshop on Distributed Event-Based Systems (DEBS), San Diego, 2003. https://www.eecg.utoronto.ca/debs03/papers/belokosztolski_etal_debs03.pdf
31. J. Bacon, D. M. Eyers, J. Singh, P. R. Pietzuch. *Access Control in Publish/Subscribe Systems.* Second International Conference on Distributed Event-Based Systems (DEBS), Rome, 2008, pp. 23-34. https://doi.org/10.1145/1385989.1385993
32. Y. Yu, A. Afanasyev, D. Clark, kc claffy, V. Jacobson, L. Zhang. *Schematizing Trust in Named Data Networking.* ACM Conference on Information-Centric Networking (ICN), 2015, pp. 177-186. https://doi.org/10.1145/2810156.2810170
33. D. Gelernter. *Generative Communication in Linda.* ACM Transactions on Programming Languages and Systems 7(1), 1985, pp. 80-112. https://doi.org/10.1145/2363.2433
34. NATS. *Auth callout.* https://docs.nats.io/running-a-nats-service/configuration/securing_nats/auth_callout
35. NATS. *Leaf nodes.* https://docs.nats.io/running-a-nats-service/configuration/leafnodes
36. NATS. *JetStream on leaf nodes.* https://docs.nats.io/running-a-nats-service/configuration/leafnodes/jetstream_leafnodes
37. NATS. *Sources and mirrors.* https://docs.nats.io/nats-concepts/jetstream/source_and_mirror
38. Kurrent (EventStoreDB). *Stream-based authorization* and *System projections.* https://docs.kurrent.io/server/v24.10/security/user-authorization.html and https://docs.kurrent.io/server/v24.10/features/projections/system.html
39. CloudEvents. *CloudEvents Specification, version 1.0.* https://github.com/cloudevents/spec/blob/main/cloudevents/spec.md
40. MOG Systems, Inc. *Selling a Place Once: Ticketing a Venue on a Replicated Log.* A companion paper. https://mog.systems/papers/selling-a-place-once/SELLING_A_PLACE_ONCE.pdf